

Citation-Aware Retrieval-Augmented Generation with Document-Grounded Answer Generation

Veenus K K

M.Tech in Machine Learning
Indian Institute of Space Science and Technology (IIST), Trivandrum

May 15, 2026

Problem Statement

Goal of the Project

To build a Citation-Aware Retrieval-Augmented Generation (RAG) system that:

- Answers user questions using external documents
- Retrieves relevant document chunks
- Generates grounded answers using an LLM
- Provides document citations along with answers
- Reduce hallucination.

Motivation

- Large Language Models may hallucinate
- Users need trustworthy answers
- Citations improve transparency and reliability

Introduction to RAG

Retrieval-Augmented Generation (RAG) combines:

- 1 Retrieval System
- 2 Large Language Model (LLM)

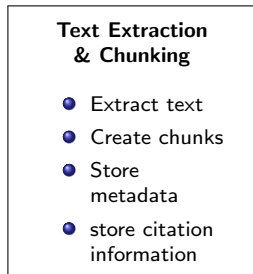
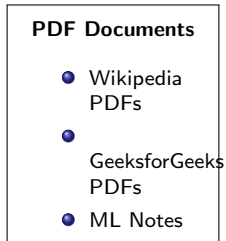
Working Principle

- Retrieve relevant document chunks
- Provide retrieved context to the LLM
- Generate grounded answers

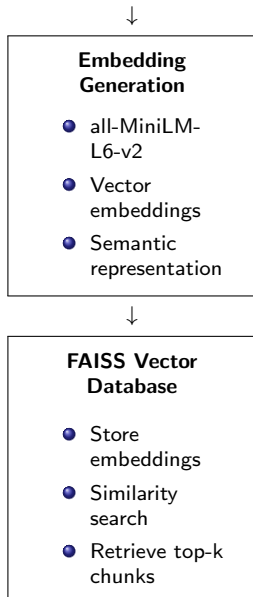
Advantages

- Reduces hallucination
- Improves factual correctness
- Enables explainability through citations

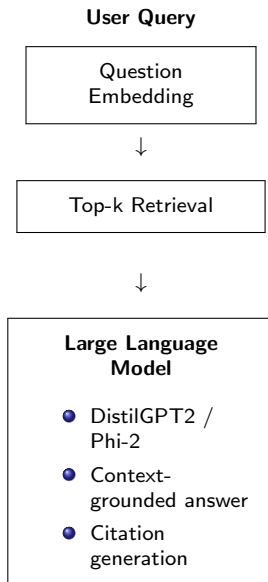
System Architecture I



System Architecture II



System Architecture III





Final Output

Answer + Citations

Example:

- Generated Answer



doc_03, Page2

Dataset Collection

Dataset Sources

- Wikipedia Articles
- GeeksforGeeks Articles
- Machine Learning Notes and PDFs

Topics Covered

- 1 linear regression
- 2 Logistic regression.
- 3 Gradient Descent
- 4 cost function
- 5 overfitting
- 6 Regularization
- 7 K-Nearest Neighbors
- 8 Support Vector Machines
- 9 decision tree
- 10 Neural Networks
- 11 Backpropagation
- 12 PCA

Text Extraction and Chunking

Text Extraction

- PDFs converted into machine-readable text
- Page-wise extraction performed
- Metadata preserved

Chunking Strategy

- Chunk Size: 400 words
- Overlap: 50 words
- Each chunk assigned metadata

Stored Metadata

- Document ID
- Page Number
- Chunk ID

Embedding Generation

Embedding Model Used

- all-MiniLM-L6-v2
- Pretrained transformer-based embedding model
- chunk embeddings is used (each chunk convert into vector)
- 384-dimensional embedding

Purpose of Embeddings

- Convert text into numerical vector representations
- Capture semantic meaning
- Enable similarity-based retrieval

Example

"Gradient descent" $\rightarrow [0.12, -0.45, 0.88, \dots]$

FAISS (Facebook AI Similarity Search)

- Stores embedding vectors
- Enables fast similarity search
- Efficient retrieval of relevant chunks

Similarity Technique Used

- Cosine Similarity
- Embedding normalization performed

Reason for Improvement

- Better semantic retrieval
- Improved relevance of retrieved chunks

Query Retrieval Process

Retrieval Pipeline

- 1 User enters query
- 2 Query converted into embedding
- 3 FAISS searches similar embeddings
- 4 Top-k chunks retrieved

Example Query

Query

What is Gradient Descent?

Retrieved Information

Gradient descent is an optimization algorithm used to minimize loss functions in machine learning.

Large Language Model Integration

LLM Used

- DistilGPT2 / Phi-2 (Experimental)

phi - 2

- phi - 2 supports a limited context length (approx. 2048 tokens).
- Hence, chunk size and top-k retrieval parameters were experimentally tuned to balance contextual coverage and retrieval noise ($k = 5$ and $\text{max_words} = 500$).
- Moderate retrieval depth produced better grounded answers.

Role of LLM

- Generate human-readable answers
- Use retrieved chunks as context
- Produce grounded responses

Prompt Engineering

- Instructed model to answer only from context
- Enforced citation-aware response generation

Final Output Format

Question

What is Gradient Descent?

Generated Answer

Gradient descent is an optimization algorithm used to minimize differentiable functions by moving in the direction of the negative gradient.

Citation

[1] doc_03, Page 1

QUESTIONS USED FOR EXPERIMENTATION

- 1 What is machine learning?
- 2 Explain about linear regression.
- 3 Explain gradient descent in machine learning.
- 4 What is the purpose of backpropagation in neural networks?
- 5 Define Support Vector Machine (SVM).
- 6 What is the role of regularization in machine learning?
- 7 Explain the working principle of K-Nearest Neighbors (KNN).
- 8 What is overfitting in machine learning models?
- 9 What are neural networks?
- 10 What is the difference between gradient descent and gradient ascent?
- 11 Why is gradient descent important in deep learning?
- 12 Explain how PCA reduce the dimension of datapoints.

Experiment 1: Effect of Top-k Retrieval

Objective

To analyze how the number of retrieved chunks affects:

- Retrieval relevance
- Answer quality
- Citation correctness
- Retrieval noise

Tested Values

- $k = 3$
- $k = 5$
- $k = 10$

Experiment 1: Effect of Top-3 Retrieval

Final Output Format

Question

Explain gradient descent in machine learning.

Generated Answer

Gradient descent is a first-order iterative algorithm for minimizing a differentiable multivariate function. It is used for optimizing a general function with the least squares loss function and Tikhonov regularization.

Sources:

doc₀₃, Page1

doc₀₆, Page4

doc₀₆, Page1

Experiment 1: Effect of Top-5 Retrieval

Question

Explain gradient descent in machine learning.

Generated Answer

Gradient descent is a method for optimizing a differentiable multivariate function. It is used for minimizing the difference between predicted and actual outputs in machine learning models.

Sources:

doc₀₃, Page1

doc₀₆, Page4

doc₁₂, Page1

doc₀₃, Page3

doc₀₆, Page1

Experiment 1: Effect of Top-10 Retrieval

Question

Explain gradient descent in machine learning.

Generated Answer

Gradient descent is a method for optimizing a differentiable multivariate function. It involves taking repeated steps in the opposite direction of the gradient of the function at the current point, because this is the direction of steepest descent. This allows us to find the minimum of the function, which is often desirable in machine learning tasks. Sources:

doc₀3, Page1

doc₀6, Page4

doc₁2, Page1

doc₀3, Page3

doc₀6, Page1

doc₀3, Page2

Experiment 1 Results

k-value	Retrieval Quality	Noise Level
2	Medium	Low
5	High	Medium
10	High	High

Observation

- Small k-values sometimes missed important context
- Large k-values introduced irrelevant chunks
- k=5 provided the best balance

Experiment 1: Example Outputs

Query: What is gradient descent?

k = 2

- Retrieved limited but relevant chunks
- Some contextual information missing

k = 5

- Better contextual coverage
- More accurate answer generation

k = 10

- Included additional irrelevant information
- Increased retrieval noise

Experiment 2: Similarity Metric Comparison

Objective

Compare retrieval performance using:

- L2 Distance
- Cosine Similarity

Method	Retrieval Performance
L2 Distance	Moderate
Cosine Similarity	slightly Better Semantic Matching

Conclusion

Cosine similarity slightly improved semantic retrieval quality.

Experiment 2: Effect on L2 Comparison I

- No normalization of embeddings
- `index = faiss.IndexFlatL2(dimension)`

Question

Define Support Vector Machine (SVM)

Retrieved Chunks

Experiment 2: Effect on L2 Comparison II

Result 1

Document: doc_08

Page: 6

Text:

Support Vector Machine (SVM) Algorithm - GeeksforGeeks

22/03/2026, 12:45

<https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/>

...

Experiment 2: Effect on L2 Comparison III

Result 2

Document: doc_08

Page: 1

Text:

Support Vector Machine (SVM) Algorithm

Last Updated: 19 Jan, 2026

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different classes in the data.

Experiment 2: Effect on L2 Comparison IV

Result 3

Document: doc_08

Page: 8

Text:

Advantages of Support Vector Machine (SVM):

- 1 High-Dimensional Performance
- 2 Nonlinear Capability using kernels such as RBF and Polynomial

Experiment 2: Effect on Cosine Similarity Comparison I

- Normalization of embeddings
- `index = faiss.IndexFlatIP(dimension)`

Question

Define Support Vector Machine (SVM).

Retrieved Chunks

Experiment 2: Effect on Cosine Similarity Comparison II

Result 1

Document: doc_08

Page: 6

Text:

Support Vector Machine (SVM) Algorithm - GeeksforGeeks

<https://www.geeksforgeeks.org/machine-learning/support-vector-machine-algorithm/>

...

Experiment 2: Effect on Cosine Similarity Comparison III

Result 2

Document: doc_08

Page: 1

Text:

Support Vector Machine (SVM) Algorithm

Last Updated: 19 Jan, 2026

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different classes in the data.

Experiment 2: Effect on Cosine Similarity Comparison IV

Result 3

Document: doc_08

Page: 8

Text:

Advantages of Support Vector Machine (SVM):

- 1 High-Dimensional Performance
- 2 Nonlinear Capability using kernels such as RBF and Polynomial

Experiment 3: Citation Analysis

Output Type	Trustworthiness	Explainability
Without Citation	Low	Low
With Citation	High	High

Observation

Citation-aware generation improved:

- Transparency
- Source verification
- Reliability of generated answers

Experimental Observations

- When irrelevant queries were asked, the LLM sometimes generated answers using pretrained internal knowledge instead of retrieved documents.
- This phenomenon is known as hallucination.
- To avoid that we can add thresholding of retrieved chunks...

```
if distances[0][0] < 0.3:  
    print("sufficiently relevant information found in the documents.")  
    exit()
```

Experimental Observations

Observations During Experiments

- Retrieval quality depends on chunk quality
- Embedding normalization improved retrieval
- Cosine similarity is better than L2 distance.
- Prompt engineering improved answer quality

Key Learning

Semantic retrieval is highly dependent on embedding quality and document preprocessing.

Challenges Faced

- Large model download time
- Retrieval mismatch for vague queries
- Chunking strategy tuning
- LLM prompt repetition
- Hardware and memory limitations

Solutions

- Used lightweight models for testing
- Applied embedding normalization
- Improved prompts
- Used cosine similarity search

Tools and Technologies Used

Component	Tool Used
Programming Language	Python
IDE	Visual Studio Code
Embeddings	Sentence Transformers
Vector Database	FAISS
LLM Framework	Hugging Face Transformers
PDF Processing	PyMuPDF
Numerical Operations	NumPy

Future Improvements

- Use larger instruction-tuned LLMs
- Improve citation accuracy
- Add reranking models
- Support multilingual documents
- Deploy as a web application

Conclusion

- Successfully implemented a Citation-Aware RAG pipeline
- Performed semantic retrieval using embeddings and FAISS
- Generated grounded answers using an LLM
- Included document-level citations in generated responses
- Improved reliability and explainability of generated answers

Final Outcome:

A working prototype of a document-grounded question answering system.

- [Hugging Face Transformers Documentation](#)
- [Sentence Transformers Documentation](#)
- [FAISS Documentation](#)
- [Wikipedia Machine Learning Articles](#)
- [GeeksforGeeks Machine Learning Articles](#)

Thank You